Growing Hypergraphs with Node Labels

Violet Ross,[1] Francis Cataldo,[2] and Philip S. Chodrow[2]

[1] *Department of Computer Science, University of Colorado at Boulder, Boulder, CO, USA*
[2] *Department of Computer Science, Middlebury College, Middlebury, VT, USA*

(Dated: June 22, 2026)

We study a mechanistic model of growing hypergraphs in which edge formation is tuned by homophily between binary node attributes. Edges form in this model as noisy copies of previous edges, where the transmission of nodes from one edge to the next depends on the labels. We derive a power law for the degree distribution in this model and describe the long-term dynamics of the joint distribution of labels contained in edges. This model defines a likelihood over a labeled hypergraph, allowing us to use standard maximum-likelihood techniques to motivate algorithms. We estimate the model parameters on synthetic and real data via (stochastic) expectation maximization, allowing us to statistically quantify the signature of homophily in empirical polyadic systems. We also demonstrate an approach to community detection via simulated annealing which, though computationally expensive, achieves competitive results on certain data sets known to be challenging to standard community detection techniques.

I. INTRODUCTION

Hypergraphs provide a natural, flexible data representation for complex systems with polyadic (multi-way) interactions, such as social interactions (both in-person [1] and online [2]), academic co-authorship [3], and ecological systems [4]. Much recent work has emphasized the importance of faithfully representing polyadic structure rather than reducing it to dyadic interactions [5, 6], though recent work has shown that some purportedly higher-order dynamics can be faithfully represented on dyadic networks under appropriate models [7].

Much attention in the theory of (hyper)graph mining has focused on developing novel techniques for community detection (also often called *clustering* or *partitioning*). Many but not all of these methods can be viewed as exact or approximate inference algorithms for stochastic generative models. A stochastic generative model is a parametrized probability distribution defined over hypergraphs. In one standard framework, a set $\mathcal{V}$ of n nodes is initialized along with a vector $\mathbf{z} \in [k]^n$ that assigns to each node one of k community labels. One then forms the edge set $\mathcal{E}$ iteratively by visiting each subset $R \subseteq \mathcal{V}$ and independently assigning a_R (possibly repeated) edges to R according to a distribution $p(a_R|\mathbf{z}_R, \theta)$, where $\mathbf{z}_R$ is the vector of labels of the nodes in R and θ is a set of model parameters. This results in a probability distribution over hypergraphs which can be factorized into a product of edge probabilities conditioned on the node labels and model parameters:

$$p(\mathcal{H}|\mathbf{z}, \theta) = \prod_{R \subseteq \mathcal{V}} p(a_R|\mathbf{z}_R, \theta) \; . \tag{1}$$

Many generative models for community detection fall into this category, including hypergraph stochastic blockmodels [8–10] and some generalizations [11, 12]. The conditional independence property eq. (1) is extremely useful for computation, as factorizability of the likelihood enables many efficiencies in various inference algorithms, including approximate maximum-likelihood estimation via direct optimization [8] or expectation-maximization [9]; spectral methods [13–15]; and message passing [16]. Another recent optimization-based approach [17] is equivalent to a microcanonical version of the stochastic blockmodel, which satisfies an asymptotic form of edge-independence in the large sparse limit. Even a recent latent-space model [18] treats edges as independent samples, albeit from a very differently stochastic generating process.

Edge independence, however, is a strong and often unrealistic assumption for modeling hypergraph data. A parallel line of work has found that polyadic data sets often contain significant edge correlations [2, 19, 20], typically in the form of *similarity between interactions*. When an interaction occurs on a set $R \subseteq \mathcal{V}$, this increases the likelihood of observing a new interaction on a set $S \subseteq \mathcal{V}$ correlated with R in the sense that $R \cap S$ is larger than would be expected by chance. These overlaps can be shown to effect the kinds of dynamics which can unfold on the hypergraph [21, 22]. Several generative models for the growth of hypergraphs with edge-dependence have been proposed [23–27]. Several of these models express dependence through a copying or recombination mechanism, in which nodes collected from one or more existing hyperedges are gathered and noisily copied into a new edge as a block.

In this paper, we introduce a model of binary node-labeled hypergraphs in which edges form via a noisy copying interaction which depends the labels. Our work is organized as follows. In Section II, we describe our model and derive limiting descriptions of the joint distribution of node labels within a given edge, as well as the degree distribution of the hypergraph. Evaluating the likelihood of our model requires marginalizing over a set of latent variables governing the noisy copy mechanism, which is computationally expensive. To estimate our model's parameters when node labels are known, we therefore describe a stochastic expectation-maximization algorithm in which each iteration has sublinear cost in the number

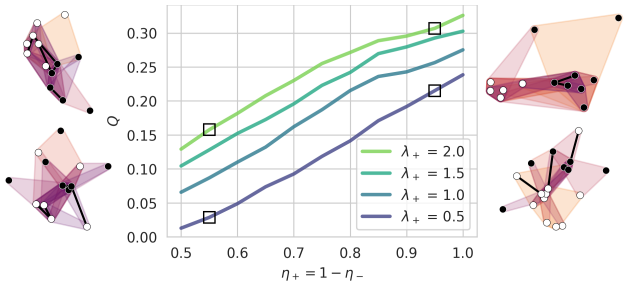

FIG. 1. Suite of model simulations illustrating the dependence of community structure, as measured by clique-projection modularity Q, on the model's parameters. The parameters $r_- = 0.5$ $a_+ = 0.2$, and $a_- = 0.1$ are held fixed. Each line corresponds to a different value of r_+ indicated by the legend. Along the horizontal axis, we vary p_+ and set $p_- = 1 - p_+$. Each parameter combination is simulated 10^2 times for 10^3 timesteps. On either side of the plot we show small hypergraphs sampled from parameter combinations indicated by the square markers. PC: Fig is good but maybe we want to do 10^3 reps just to reduce the noise a bit, and possibly increase the resolution of the copy parameters.

of edges for sparse hypergraphs. We also describe a community detection algorithm for estimating node labels via simulated annealing. In Section IV we demonstrate our algorithms on synthetic and empirical data. We show that, on synthetic data sampled from our model, our stochastic expectation maximization algorithm is both consistent and scalable, and that our simulated annealing algorithm for community detection, though computationally expensive, outperforms other standard methods. On empirical data, our parameter estimatates yield insight into the mechanisms governing edge formation Our community detection achieves competitive recovery of observed metadata labels on some data sets known to be challenging to other community detection methods. We conclude with some discussion of future directions for models of node-labeled hypergraphs with edge-dependence.

II. MODEL DESCRIPTION

Our model builds on the Hyperedge Copy Model (HCM) of He et al. [25]. In their model, edges form through a combination of three distinct steps: a copy step, in which a previous edge is sampled and imperfectly copied; an extant node addition step, in which new nodes are added to the new edge from the existing node set; and a new node addition step, in which new nodes are added to the new edge from outside the existing node set. Our model replicates these mechanisms but adds label-aware parameters to regulate each.

Formally, a state of our model at time t is a hypergraph $\mathcal{H}^{(t)} = \left(\mathcal{V}^{(t)}, \mathcal{E}^{(t)}\right)$, where $\mathcal{V}$ is the set of nodes and $\mathcal{E}$ is the set of hyperedges. Each node $v \in \mathcal{V}$ has a binary label $z_v \in \{0, 1\}$. We let $\bar{z} = 1 - z$ denote the opposite label of z. At timestep t, add edge e to $\mathcal{E}^{(t)}$ to obtain $\mathcal{E}^{(t+1)}$, where e is formed according to the following three-step process:

- **Edge-Copying**: An edge f is sampled uniformly at random from $\mathcal{E}^{(t-1)}$, and a node u is sampled uniformly at random from f. Then, each node v in f is included in e with probability p_+ if $z_v = z_u$ and with probability p_- if $z_v \neq z_u$.

- **Extant Node Addition**: We sample X_{z_u} nodes from $\mathcal{V} \setminus f$ with label z_u and add them to e, where X_z is a Poisson random variable with mean r_+. We similarly sample $X_{\bar{z}_u}$ nodes from $\mathcal{V} \setminus f$ with label $\bar{z}_u$ and add them to e, where $X_{\bar{z}_u}$ is a Poisson random variable with mean r_-.

- **Novel Node Addition**: We sample Y_{z_u} new nodes with label z_u and add them to e, where Y_z is a Poisson random variable with mean a_+. We similarly sample $Y_{\bar{z}_u}$ new nodes with label $\bar{z}_u$ and add them to e, where $Y_{\bar{z}_u}$ is a Poisson random variable with mean a_-.

In our simulations we initialize $\mathcal{H}^{(0)}$ with a single edge containing one node of each label, but any combination of hypergraph and node labels can be used as an initial condition. Both our simulations and our statistical inference techniques (described below) were carried using the XGI package for polyadic data analysis in Python [28].

Figure 1 illustrates role of the model parameters in tuning the community structure of realized hypergraphs. We measure community structure by computing Newman-Girvan modularity [29] on the clique-projection of the hypergraph. For fixed r_-, a_+, and a_-, increasing the difference $p_+ - p_-$ leads to higher modularity, as does increasing r_+. Hypergraphs sampled from models with high values of p_+ and r_+ tend to have label-homogeneous cores of densely-overlapping hyperedges, which are weakly connected to one another by edges with more heterogeneous label compositions. Hypergraphs with lower values of p_+ and r_+ tend to have less-structured distributions of labels over hyperedges.

A. Asymptotic Properties

1. *Joint Distribution of Labels in Edges*

The asymptotics of this model can be described in terms of the joint distribution $p_{K_0,K_1}(k_0, k_1)$ of nodes of labels 0 and 1 in a uniformly random edge. This joint distribution can be obtained as the leading eigenvector of a linear map. Let $p^{(t)}_{K_0,K_1}(k_0, k_1)$ be the joint distribution of label counts in a uniformly random edge after t timesteps. Then, the probability $q(k_0', k_1')$ of realizing an edge with k_0' nodes of label 0 and k_1' nodes of label 1 at

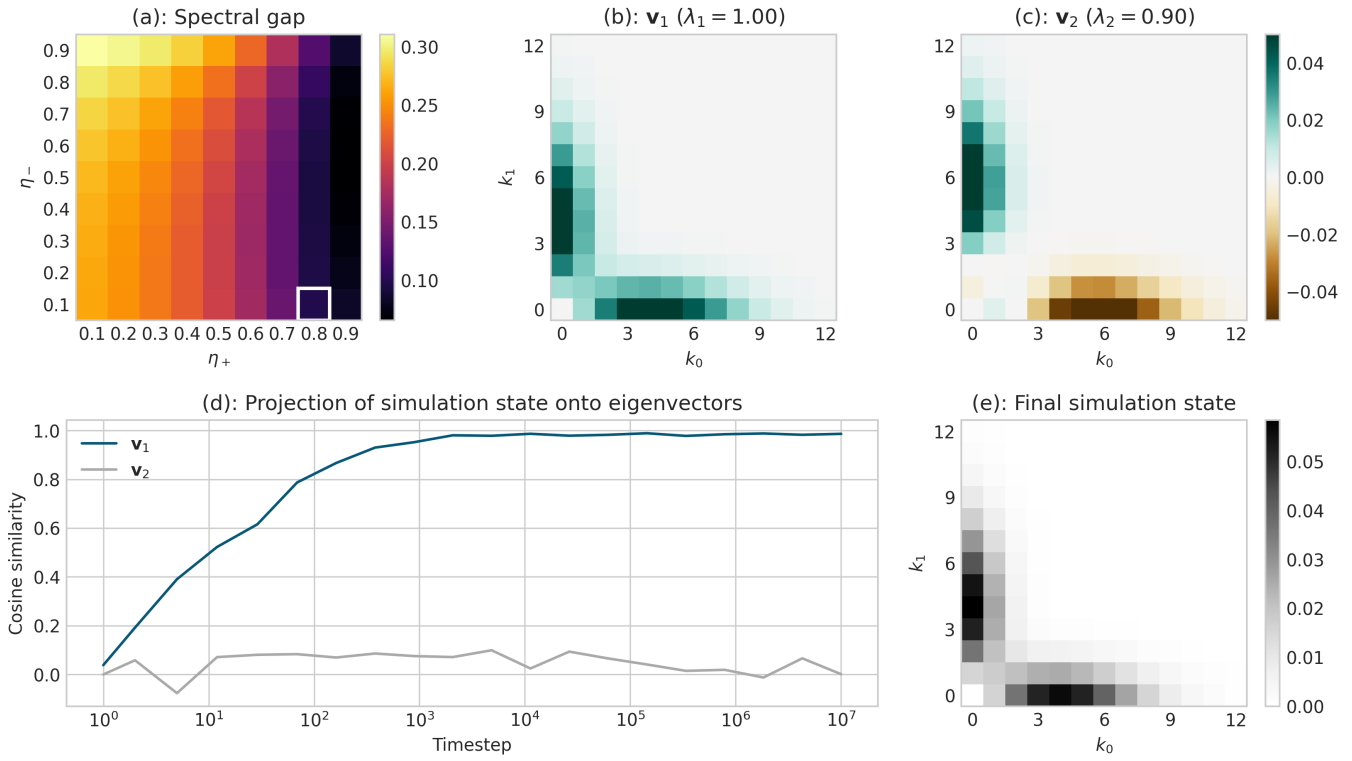

FIG. 2. Visualization of the long-run dynamics of our model, for parameters. (a): Spectral gap $1-\lambda_2$ of the linear map T as a function of p_+ and p_- for fixed parameter values $r_+ = 0.5$, $r_- = a_-3$, $a_+ = 0.4$ and $a_- = 0.2$. The highlighted values $p_+ = 0.8$ and $p_- = 0.1$ are used in the remainder of the visualization. (b): Leading eigenvector $\mathbf{v}_1$ of T, describing the stationary joint distribution $p^{(\infty)}_{K_0,K_1}(k_0,k_1)$ of label counts in edges. (c): Second eigenvector $\mathbf{v}_2$ of T. The transients of the model dynamics are dominated by the subspace spanned by $\mathbf{v}_1$ and $\mathbf{v}_2$. (d): Euclidean projections of the simulated joint distribution $p^{(t)}_{K_0,K_1}(k_0,k_1)$ onto the eigenvectors $\mathbf{v}_1$ and $\mathbf{v}_2$ during a simulation run with 10^7 timesteps. At each time point, we aggregate the most recent $1,000$ edges to compute the joint distribution in order to approximate an "instantaneous" simulation state. (e): Empirical joint distribution $p^{(t)}_{K_0,K_1}(k_0,k_1)$ aggregated across 10^7 timesteps.

time $t+1$ is given by

$$q(k_0',k_1') = \sum_{k_0,k_1} T(k_0',k_1'|k_0,k_1)p^{(t)}_{K_0,K_1}(k_0,k_1)\ ,$$

where $T(k_0',k_1'|k_0,k_1)$ gives the probability that the edge e realized in step $t+1$ has k_0' nodes of label 0 and k_1' nodes of label 1 given that the edge f sampled to form e has k_0 nodes of label 0 and k_1 nodes of label 1. At stationarity, we must have $q(k_0',k_1') = p^{(t)}_{K_0,K_1}(k_0',k_1') = p^{(t+1)}_{K_0,K_1}(k_0',k_1')$, so the stationarity condition can be obtained by solving the linear system

$$p^{(t+1)}_{K_0,K_1}(k_0',k_1') = \sum_{k_0,k_1} T(k_0',k_1'|k_0,k_1)p^{(t)}_{K_0,K_1}(k_0,k_1)\ .$$

The stationary joint distribution $p^{(\infty)}_{K_0,K_1}(k_0,k_1)$ is therefore given by the leading eigenvector of the doubly-indexed matrix T with entries $T(k_0',k_1'|k_0,k_1)$. The entries of this matrix can be computed analytically. Conditional on choosing an edge f with k_0 nodes of label 0 and k_1 nodes of label 1, a node of label z is selected as the seed node u with probability $\frac{k_z}{k_0+k_1}$. Then, the number of nodes of labels z and $\bar{z}$ in e have distributions

$$k_z' \sim 1 + \text{Binom}\,(k_z - 1, p_+) + \text{Poisson}(r_+ + a_+)\ ,$$
$$k_{\bar{z}}' \sim \text{Binom}\,(k_{\bar{z}}, p_-) + \text{Poisson}(r_- + a_-)\ .$$

The probability of obtaining k_0' nodes of label 0, conditioning on k_0 and k_1, is

$$t(k_0'|k_0,k_1) = \frac{k_0}{k_0+k_1}\sum_{i=0}^{k_0'-1}\binom{k_0-1}{i}p_+^i(1-p_+)^{k_0-1-i}e^{-(r_++a_+)}\frac{(r_++a_+)^{k_0'-1-i}}{(k_0'-1-i)!} + \frac{k_1}{k_0+k_1}\sum_{i=0}^{k_0'}\binom{k_0}{i}p_-^i(1-p_-)^{k_0-i}e^{-(r_-+a_-)}\frac{(r_-+a_-)^{k_0'-i}}{(k_0'-i)!}\ .$$

A parallel expression describes $t(k_1'|k_0,k_1)$, and the transition matrix entries are given by the product

$$T(k_0',k_1'|k_0,k_1) = t(k_0'|k_0,k_1)t(k_1'|k_0,k_1)\ .$$

In Figure 2, we study the dynamics of our model through the matrix T. For fixed values of the parameters r_+, r_-, a_+ and a_-, the spectral gap $1-\lambda_2$ of T is highest when p_- is large and p_+ is small. High values of the spectral gap correspond to fast convergence to the stationary distribution $p^{(\infty)}_{K_0,K_1}(k_0,k_1)$ given by the Perron eigenvector $\mathbf{v}_1$ of T with eigenvalue 1 (panel (b)). We instead illustrate an alternative regime of relatively slow convergence in which p_+ is large and p_- is small. In this regime, the simulation may maintain transient dynamics in the direction of the second eigenvector $\mathbf{v}_2$ of T with eigenvalue $\lambda_2 < 1$ (panel (c)) for relatively long periods of time before converging to the stationary distribution. We illustrate this behavior in panel (d), where the system converges relatively slowly to the leading eigenvector $\mathbf{v}_1$, resulting in a final simulation that matches it closely (panel (e)).

Figure 2 shows that, when p_+ is sufficiently large, the stationary distribution $p^{(\infty)}_{K_0,K_1}(k_0,k_1)$ is concentrated on edges in which typical edges have large majorities of nodes with the same label (panel b), thus displaying a form of higher-order homophily [30]. The stationary distribution is symmetric, implying that neither node label is favored in the long-run. The second eigenvector v_2 (panel c), however, is antisymmetric, describing a transient in which one a single label tends to dominate most edges. When p_+ is sufficiently large, the spectral gap $1-\lambda_2$ is small (panel a), indicating that this transient decays slowly and that the system may be dominated by edges with a single majority label for relatively long periods of time before equilibrating to the leading eigenvector (panels d and e).

2. *Degree Distribution*

The degree distribution of a hypergraph generated from our model follows an asymptotic power law with an exponent which can be computed in closed form from the model parameters and the stationary joint distribution $p^{(\infty)}_{K_0,K_1}(k_0,k_1)$. Our derivation follows that of He et al. [25], which in turn is based on a derivation by Mitzenmacher [31]. To carry out the argument, we require a mean-field assumption that, when a node v is selected from edge f in the edge-sampling step, the degree d of v is independent of the ratio $\frac{k_0}{k}$ of label-0 nodes in the edge f sampled to form the new edge e.

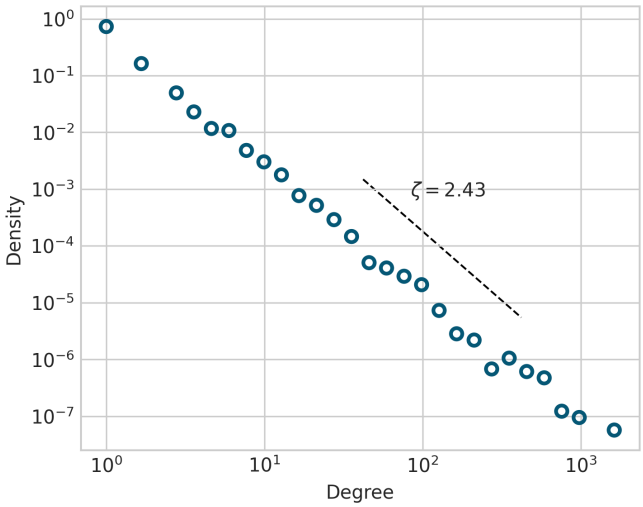

FIG. 3. Degree distribution from model simulation of 10^4 edges (points) and theoretical power law (line) with exponent given by Equation (2). The parameters in this experiment are $p_+ = 0.7, p_- = 0.3, r_+ = 0.5, r_- = a_-3, a_+ = 0.2, a_- = 0.1$.

Under this assumption, it is possible to show that the degree distribution follows a power law with exponent

$$\zeta = 1 + \frac{\langle k\rangle}{1 + p_+(\rho_{00}+\rho_{11}-1) + 2p_-\rho_{01}} \qquad (2)$$
$$\rho_{ij} = \left\langle \frac{k_i k_j}{k} \right\rangle\ .$$

In the expression for ρ_{ij} (and for $\langle k\rangle$) the expectation is taken with respect to the stationary joint distribution $p^{(\infty)}_{K_0,K_1}(k_0,k_1)$. Here, a can be interpreted as the expected number of novel nodes added per new edge, while σ can be interpreted as the expected number of nodes copied into a new edge from the sampled edge, normalized by the average degree of a node in the hypergraph.

3. *Community Structure*

Show a plot describing how the modularity of the clique projection varies by parameter, and perhaps some sample

hypergraphs with associated partitions.

III. MODEL INFERENCE

A. Parameter Estimation via Stochastic EM

We first consider the problem of learning the parameter vector $\boldsymbol{\theta} = (p_+, p_-, r_+, r_-, a_+, a_-)$ given an observed hypergraph $\mathcal{H}$ with sequential edge indices and known node labels $\mathbf{z}$. We aim to do this via maximum-likelihood estimation. Computing the complete data likelihood requires marginalizing over the model latent variables, which in this case are the edge f and node u selected at each timestep of the growth model to form the new edge e. For $e \in \mathcal{E}$, let t_e denote the time at which e was added to the hypergraph. We say that $f \prec e$ if $t_f < t_e$. Then, the complete data likelihood can be written

$$\mathbb{P}(\mathcal{H}, \mathbf{z}|\boldsymbol{\theta}) = \prod_{e\in\mathcal{E}} \sum_{f\prec e} \sum_{u\in f} p(e|f, u, \mathbf{z}, \boldsymbol{\theta}) p(f, u|\mathcal{E}_{t-1}, \mathbf{z}, \boldsymbol{\theta}) \ . \tag{3}$$

For notational simplicity, we have repressed the additional entries of $\mathbf{z}$ introduced by the novel node addition step. The requirement to sum over the latent variables at each timestep implies a computational cost to evaluate $\mathbb{P}(\mathcal{H}, \mathbf{z}|\boldsymbol{\theta})$ (or its derivatives) which is quadratic in the number of edges, making direct optimization of the likelihood expensive for data sets of even modest size. Batch expectation-maximization [32] is an alternative approach which maximizes the likelihood without explicitly computing eq. (3), but even this approach requires forming a belief distribution over the values of all latent variables at each timestep, which again incurs quadratic cost.

We therefore use a stochastic expectation-maximization (SEM) algorithm [33] to estimate the parameters $\boldsymbol{\theta}$. Each iteration of the SEM algorithm consists of two steps: the E-step, in which exponentially-weighted moving averages of the sufficient statistics are calculated using the current parameter estimates, and the M-step, in which the parameter estimates are updated using the expected sufficient statistics.

To initialize the SEM algorithm, we initialize a vector $\mathbf{s}^{(0)}$ of sufficient statistics to arbitrary values; in our model there is a sufficient statistic for each parameter. We then update a moving average of $\mathbf{s}$ as follows. In each algorithmic step ℓ, we pick an edge e uniformly at random from $\mathcal{H}$. Then, given current estimates $\hat{\boldsymbol{\theta}}^{(\ell)}$, we form a belief distribution over the latent identities of $f \in \mathcal{E}$ and $u \in f$ which could have been selected in the edge-copying step to form e. Given e, the probability that f and u were selected can be computed in closed form. Let $m^{(t-1)}$ denote the size of the edge set at time $t-1$.

We have

$$p(f, u|\mathcal{E}^{(t-1)}, \mathbf{z}, \hat{\boldsymbol{\theta}}^{(\ell)}) = \frac{1}{|f|\, m^{(t-1)}} \triangleq r_{t-1}(f) \ ,$$

so Bayes' rule gives

$$p(f, u|e, \hat{\boldsymbol{\theta}}^{(\ell)}) = \frac{p(e|f, u, \mathbf{z}, \hat{\boldsymbol{\theta}}^{(\ell)}) r_{t-1}(f)}{\sum_{f'\prec e} \sum_{u'\in f'} p(e|f', u', \mathbf{z}, \hat{\boldsymbol{\theta}}^{(\ell)}) r_{t-1}(f')} \ . \tag{4}$$

A helpful note for computation is that $p(e|f, u, \mathbf{z}, \hat{\boldsymbol{\theta}}^{(\ell)}) = 0$ if either $u \notin e \cap f$ or $f \not\prec e$. We then form the vector of expected sufficient statistics

$$s' = \sum_{f\prec e} \sum_{u\in f\cap e} p(f, u|e, \hat{\boldsymbol{\theta}}^{(\ell)}) \sigma(e, f, u, \mathbf{z}) \ , \tag{5}$$

where $\sigma(e, f, u, \mathbf{z})$ is the vector of sufficient statistics for the parameters $\boldsymbol{\theta}$ given e, f, u, and $\mathbf{z}$. The entries of $\sigma(e, f, u, \mathbf{z})$ can be computed as functions of the node labels in e and f; we supply explicit formulae in Section B. We then perform the update

$$\mathbf{s}^{(\ell+1)} = (1 - \alpha^{(\ell)})\mathbf{s}^{(\ell)} + \alpha^{(\ell)}\mathbf{s}' \ ,$$

where $\alpha^{(\ell)}$ is the learning rate at step ℓ; we use an exponentially decaying learning rate in our experiments. Our estimate of the parameter vector $\boldsymbol{\theta}$ is then given by a function $\hat{\boldsymbol{\theta}}^{(\ell+1)} = g(\mathbf{s}^{(\ell+1)})$ of the sufficient statistics, which we also supply in Section B. Algorithm 1 summarizes the steps of this algorithm. Our version of the algorithm uses an online computation of $\mathbf{s}'$, which allows us to perform the double-sum appearing in eqs. (4) and (5) only once, and without explicitly storing the resulting belief distribution at any point.

Algorithm 1 SEM Update Step

Require: $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, $\mathbf{z}$, $\hat{\boldsymbol{\theta}}^{(\ell)}$, $\mathbf{s}^{(\ell)}$
  $e \leftarrow \text{Uniform}(\mathcal{E})$
  $a \leftarrow \mathbf{0}$
  $b \leftarrow \mathbf{0}$
  for $f \in \mathcal{E}$ **do**
    if $f \prec e$ **then**
      for $u \in e \cap f$ **do**
        $a \leftarrow a + p(f, u|e, \hat{\boldsymbol{\theta}}^{(\ell)})\sigma(e, f, u, \mathbf{z})$
        $b \leftarrow b + p(f, u|e, \hat{\boldsymbol{\theta}}^{(\ell)})$
  $\mathbf{s}' \leftarrow a/b$
  $\mathbf{s}^{(\ell+1)} \leftarrow (1 - \alpha^{(\ell)})\mathbf{s}^{(\ell)} + \alpha^{(\ell)}\mathbf{s}'$
  $\hat{\boldsymbol{\theta}}^{(\ell+1)} \leftarrow g(\mathbf{s}^{(\ell+1)})$
  return $\hat{\boldsymbol{\theta}}^{(\ell+1)}$, $\mathbf{s}^{(\ell+1)}$

The SEM algorithm continues iterating until the stopping criterion is met. In our experiments, the stopping criterion is met when each of the current parameter estimates $\hat{\boldsymbol{\theta}}^{(\ell)}$ is within [THRESH] of the [LENGTH] preceding estimates of the corresponding parameter.

B. Community Detection via MLE

Equation (3) defines a likelihood function over both the parameter vector $\boldsymbol{\theta}$ and the node labels $\mathbf{z}$. This allows

us to attempt to also learn $\mathbf{z}$ via maximum-likelihood estimation. Unlike in stochastic blockmodel variants in which there are several heuristics with favorable computational properties [8, 9, 12, 13, 15, 17, 34], the dependence structure encoded by our model makes it difficult to apply standard efficient heuristics; the development of such heuristics would be an important direction of future work. In this study, we therefore attempt to learn $\mathbf{z}$ via simulated annealing [35], which is a general-purpose optimization algorithm that can be used to approximately maximize the likelihood over $\mathbf{z}$. Our primary aim is to demonstrate that likelihood maximization in this model can detect label structure in at least some circumstances where more standard community detections may fail, thereby motivating the development of more efficient optimization heuristics in future work.

A direct evaluation of eq. (3) requires $O(m^2)$ terms, making it expensive to evaluate even for data sets of modest size. An important computational limitation of simulated annealing in this context is that the label z_i of node i appears in *all* terms of the likelihood eq. (3) corresponding to edges after which node i arrives. This includes edges in which node i does not participate, implying that we must perform $O(m^2)$ computations even to evaluate the change in likelihood under a proposed change to z_i.

To mitigate the computational burden, we use an intersection-based approximation to the likelihood of forming edge e in which we only sum over candidate copied edges f such that $f = \text{argmax}_{f' \in \mathcal{E}} |f' \cap e|$. To justify this approximation, we start by observing that, if f is the edge sampled to form e, then the number of extant nodes required to form e is $x = |e| - |e \cap f| - |e \setminus \mathcal{V}|$. We'll argue that, when the number of nodes n is large, $p(e|f, u, \mathbf{z}, \boldsymbol{\theta}) \in O(n^{-x})$, which implies that edges f which do not maximize $|f \cap e|$ will have negligible contributions to the likelihood. The essence of the argument is that forming x extant nodes requires x independent samples from the sets $\mathcal{V}_0 \setminus f$ and $\mathcal{V}_1 \setminus f$, each of which have size scaling linearly with n. So, each independent sample has probability $O(n^{-1})$, and the probability of forming a given set of x extant nodes is $O(n^{-x})$. We detail this argument in Section C.

In our application of simulated annealing, we run the algorithm for $A \times n$ steps where A represents the number of epochs. In each step, we propose a change to the labels $\mathbf{z}$ by generating a node index j uniformly at random and evaluating the change in likelihood of "flipping" the binary node label ($\mathbf{z}_j = 1 - \mathbf{z}_j$). If the proposal increases Equation (C9), it is accepted. However, if the proposal decreases the likelihood, we accept it with a probability that decreases as the epochs increase. When evaluating this change in likelihood, we implement custom data structures that allow us to compute only the change in likelihood from the prior state by storing the components of the prior likelihood calculation, resulting in a 30 times decrease in compute time on real world datasets at the expense of memory.

In simulated annealing applications, it is important accept proposals that decrease the value of the objective function with a probability that decreases gradually as the algorithm runs. Similarly, proposals that slightly lower the objective function should be accepted more frequently than proposals causing large objective function decreases. A balance must be struck, both accepting enough proposals decreasing the objective function to avoid becoming stuck in local optima and not accepting too many to allow convergence to a maxima. However, the change in log-likelihood of a proposal in our application changes significantly depending on the size of the hypergraph considered. Thus, traditional annealing schedules struggle to accept an effective balance of proposals that decrease the likelihood on varying sized hypergraphs. For example, on datasets with large m, the logged form of Equation (C9) have orders of magnitude larger changes compared to hypergraphs with small m. Thus, we implement a custom annealing schedule based on the idea of z-scores. Breaking the runtime into epochs, the algorithm's first epoch accepts all proposals while storing the changes in log-likelihood and, at the end of the first epoch, calculating its standard deviation, $\sigma_{\mathcal{H}}$. For the rest of the steps of the algorithm, the acceptance probability of a proposal that decreases the likelihood is determined by the change in log-likelihood of the proposal divided by $\sigma_{\mathcal{H}}$, interpreting the change in likelihood in context of the specific hypergraph. A full specification of the algorithm is provided in Section D.

FC: Need data structure description??

IV. RESULTS

How did we do on real and synthetic data?

We evaluate our methods on several data sets.

Contact high school: [1, 2] Contact primary school: [2, 36, 37] Senate bills: [2, 38, 39] Coauthorship: ??? PC: Violet can you supply the reference here

A. Simulated Annealing for Community Detection

We evaluate our simulated annealing algorithm on a range of small synthetic data sets under two conditions, shown in Figure 4. Under the Nishimori condition [40], we generate a synthetic hypergraph according to a parameter vector $\boldsymbol{\theta}$ and then attempt to learn $\mathbf{z}$ via simulated annealing using the same parameter vector $\boldsymbol{\theta}$. Under the "fixed" condition, which simulates a more realistic data analysis scenario, we always use the same parameter vector $\boldsymbol{\theta}'$ to learn $\mathbf{z}$ via simulated annealing, regardless of the true parameter vector $\boldsymbol{\theta}$ used to generate the synthetic data. In this condition, we use $\boldsymbol{\theta}' = (0.9, 0.1, 0.2, 0.1, 2.0, 0.3)$, which is the same parameter vector used in Figure 2 and which we found to be a reasonable default for learning $\mathbf{z}$ across a range of synthetic data sets generated with different parameters. We

TABLE I. SEM on empirical data

	$\hat{p}_+$	$\hat{p}_-$	$\hat{r}_+$	$\hat{r}_-$	$\hat{a}_+$	$\hat{a}_-$
senate-bills						
coauthorship						
primary-school						
high-school						

compare these methods against each other and against a baseline of Laplacian spectral clustering applied to the weighted clique projection of the hypergraph.

Comparing the first and second rows of Figure 4, we find that in these examples the penalty for using the "wrong" parameter vector $\boldsymbol{\theta}'$ is relatively small. Furthermore, maximum-likelihood estimation under both the Nishimori and fixed conditions tends to outperform clique-projection spectral clustering in most of the areas of parameter space explored by this experiment.

B. Parameter Estimation

Results of SEM on empirical data are in Table I.

p_+		p_-		r_+		r_-		a_+		a_-	
v	ε	v	ε	v	ε	v	ε	v	ε	v	ε
0.900	+0.005	0.900	-0.014	0.200	+0.009	0.200	-0.032	0.200	+0.002	0.200	-0.043
0.200	-0.011	0.200	+0.002	2.000	-0.060	2.000	+0.134	0.200	-0.012	0.200	-0.009
0.200	-0.005	0.200	+0.045	0.200	+0.016	0.200	-0.035	2.000	-0.073	2.000	-0.031
0.900	+0.001	0.900	+0.001	2.000	-0.003	2.000	-0.143	2.000	-0.109	2.000	+0.191
0.400	-0.007	0.400	-0.001	0.700	-0.022	0.700	-0.014	0.700	-0.068	0.700	-0.016
0.800	-0.017	0.400	+0.011	0.200	-0.020	0.200	+0.015	0.200	+0.029	0.200	-0.035
0.200	+0.005	0.200	+0.006	1.000	+0.023	0.500	-0.011	0.200	-0.001	0.200	+0.017
0.200	+0.013	0.200	+0.007	0.200	-0.014	0.200	+0.003	1.000	-0.048	0.500	+0.045
0.600	+0.011	0.400	+0.012	1.400	+0.011	1.000	-0.057	0.800	-0.002	0.200	-0.019
0.700	+0.004	0.620	-0.012	1.000	-0.051	0.800	+0.047	0.300	-0.020	0.250	-0.045
0.400	-0.015	0.800	-0.012	0.200	+0.004	0.200	+0.001	0.200	-0.042	0.200	-0.002
0.200	-0.016	0.200	+0.010	0.500	+0.005	1.000	+0.007	0.200	-0.000	0.200	+0.018
0.200	-0.008	0.200	-0.012	0.200	-0.021	0.200	+0.009	0.500	+0.021	1.000	+0.025
0.400	+0.008	0.600	-0.013	1.000	+0.001	1.400	-0.031	0.200	+0.020	0.800	-0.031
0.620	+0.012	0.700	-0.023	0.800	+0.058	1.000	-0.114	0.250	-0.039	0.300	-0.007

TABLE II. True parameter values and SEM estimation error ($\varepsilon = \hat{\theta} - \theta$) for each graph.

VR: Hate the presentation of this (table 2). Thinking of better ways to convey this information.

V. DISCUSSION

We have proposed a simple model of binary-labeled hypergraph formation in which edges are formed through noisy copies of previous edges, combined with the addition of extant and novel nodes. Our model incorporates label homophily through each of the three edge formation mechanisms.

PC: More discussion

There are several directions for future work. Most obviously, although our results show that maximum-likelihood inference is a reasonable approach for community detection in this model, the simulated annealing method we have used is impractically expensive on data sets of even relatively modest size. Developing more efficient heuristics for maximizing the likelihood over $\mathbf{z}$ will be an important direction for scalable application of our proposed model.

There are also opportunities to extend the model to more realistic settings and more diverse data sets. One obvious extension is to nonbinary node-labels, which would enable multiway community detection algorithms. Another approach is to relax the assumption in our model that *every* edge is formed through the edge-copying mechanism; one could instead allow some edges to form spontaneously without copying an earlier edge. A final obvious extension is to incorporate recency-bias [23] into the copy mechanism, so that more recent edges are more likely to be copied while edges which are sufficiently old may cease to be copied at all.

ACKNOWLEDGMENTS

PSC acknowledges support from the National Science Foundation under DMS-2407058. Compute was provided by Middlebury College through support from the National Science Foundation under award 1827373.

DATA AVAILABILITY

The code used to generate the results in this paper is available at [GITHUB LINK]. The empirical data sets analyzed in this paper are available at the following links:

- Contact high school:
- Contact primary school:
- Senate bills:
- Coauthorship:

[1] R. Mastrandrea, J. Fournet, and A. Barrat, PloS one **10**, e0136497 (2015).

[2] A. R. Benson, R. Abebe, M. T. Schaub, A. Jadbabaie,

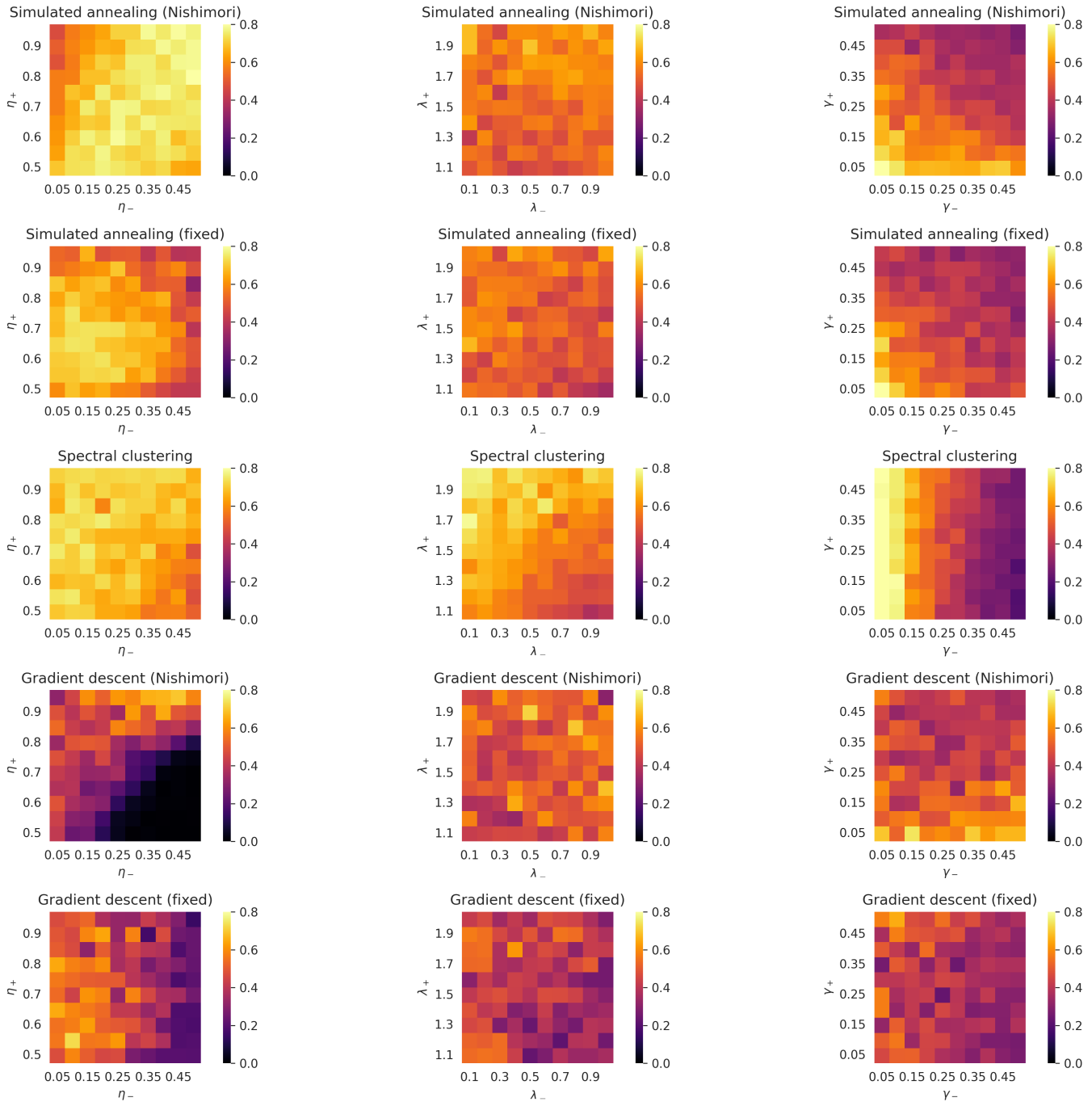

FIG. 4. Community detection results on synthetic data under the Nishimori condition [40] of exactly known parameters. Each pixel represents 200 independent hypergraphs generated from our model with 100 timesteps. The default parameters are $(p_+, p_-, r_+, r_-, a_+, a_-) = (0.9, 0.1, 2.0, 0.3, 0.2, 0.1)$; in each panel, two of these parameters instead vary across the range shown on the axes. PC: This figure is probably a bit too bulky in comparison to the value it adds, consider ways to edit, compactify, or move part of it to supplementary.

and J. Kleinberg, Proceedings of the National Academy of Sciences **115**, E11221 (2018).
[3] H. Wu, Y. Yan, and M. K.-P. Ng, IEEE Transactions on Pattern Analysis and Machine Intelligence **45**, 3245 (2022).
[4] J. M. Levine, J. Bascompte, P. B. Adler, and S. Allesina, Nature **546**, 56 (2017).
[5] F. Battiston, G. Cencetti, I. Iacopini, V. Latora, M. Lucas, A. Patania, J.-G. Young, and G. Petri, Physics Reports **874**, 1 (2020).
[6] C. Bick, E. Gross, H. A. Harrington, and M. T. Schaub, SIAM review **65**, 686 (2023).

[7] T. P. Peixoto, L. Peel, T. Gross, and M. De Domenico, arXiv preprint arXiv:2602.16937 (2026).
[8] P. S. Chodrow, N. Veldt, and A. R. Benson, Science Advances **7**, eabh1303 (2021).
[9] N. Ruggeri, M. Contisciani, F. Battiston, and C. De Bacco, Science Advances **9**, eadg9159 (2023).
[10] C. Kim, A. S. Bandeira, and M. X. Goemans, arXiv preprint arXiv:1807.02884 (2018).
[11] A. Badalyan, N. Ruggeri, and C. De Bacco, Nature Communications **15**, 7073 (2024).
[12] J. Hood, C. De Bacco, and A. Schein, Nature Communications (2026).
[13] P. S. Chodrow, N. Eikmeier, and J. L. Haddock, SIAM Journal on Mathematics of Data Science **5**, 251 (2023).
[14] M. Fernandez V, L. Stephan, and Y. Zhu, arXiv preprint arXiv:2604.20907 (2026).
[15] J. Li, M. T. Schaub, and L. Peel, arXiv preprint arXiv:2601.10502 (2026).
[16] N. Ruggeri, A. Lonardi, and C. De Bacco, Journal of Statistical Mechanics: Theory and Experiment **2024**, 043403 (2024).
[17] J. Kritschgau, D. Kaiser, O. Alvarado Rodriguez, I. Amburg, J. Bolkema, T. Grubb, F. Lan, S. Maleki, P. S. Chodrow, and B. Kay, Scientific Reports **14**, 6933 (2024).
[18] X. Yu and J. Zhu, Journal of the American Statistical Association **120**, 1491 (2025).
[19] N. W. Landry, J.-G. Young, and N. Eikmeier, EPJ Data Science **13**, 17 (2024).
[20] G. Lee, M. Choe, and K. Shin, in *Proceedings of the Web Conference 2021* (ACM, Ljubljana Slovenia, 2021) pp. 3396–3407.
[21] F. Malizia, S. Lamata-Otín, M. Frasca, V. Latora, and J. Gómez-Gardeñes, Nature Communications **16**, 555 (2025).
[22] S. Lamata-Otín, F. Malizia, V. Latora, M. Frasca, and J. Gómez-Gardeñes, Physical Review E **111**, 034302 (2025).
[23] A. R. Benson, R. Kumar, and A. Tomkins, in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (ACM, London United Kingdom, 2018) pp. 1148–1157.
[24] C. Avin, Z. Lotker, Y. Nahum, and D. Peleg, in *Proceedings of the 2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining* (ACM, Vancouver British Columbia Canada, 2019) pp. 398–405.
[25] X. He, P. S. Chodrow, and P. J. Mucha, Physical Review E **112**, 024305 (2025).
[26] G. Lee and K. Shin, in *2021 IEEE International Conference on Data Mining (ICDM)* (IEEE, Auckland, New Zealand, 2021) pp. 310–319.
[27] G. Lee and K. Shin, Knowledge and Information Systems **65**, 1549 (2023).
[28] N. W. Landry, M. Lucas, I. Iacopini, G. Petri, A. Schwarze, A. Patania, and L. Torres, Journal of Open Source Software **8**, 5162 (2023).
[29] M. E. Newman and M. Girvan, Physical review E **69**, 026113 (2004).
[30] M. Laber and B. Klein, arXiv preprint arXiv:2606.02537 (2026).
[31] M. Mitzenmacher, Internet Mathematics **1**, 226 (2004).
[32] A. P. Dempster, N. M. Laird, and D. B. Rubin, Journal of the Royal Statistical Society: Series B (Methodological) **39**, 1 (1977).
[33] O. Cappé and E. Moulines, Journal of the Royal Statistical Society Series B: Statistical Methodology **71**, 593 (2009).
[34] M. E. Newman, Physical Review E **94**, 052315 (2016).
[35] S. Kirkpatrick, C. D. Gelatt Jr, and M. P. Vecchi, Science **220**, 671 (1983).
[36] J. Stehlé, N. Voirin, A. Barrat, C. Cattuto, L. Isella, J.-F. Pinton, M. Quaggiotto, W. Van den Broeck, C. Régis, B. Lina, *et al.*, PloS one **6**, e23176 (2011).
[37] V. Gemmetto, A. Barrat, and C. Cattuto, BMC infectious diseases **14**, 695 (2014).
[38] J. H. Fowler, Political analysis **14**, 456 (2006).
[39] J. H. Fowler, Social networks **28**, 454 (2006).
[40] A. Decelle, F. Krzakala, C. Moore, and L. Zdeborová, Physical Review E—Statistical, Nonlinear, and Soft Matter Physics **84**, 066106 (2011).
[41] A. A. Hagberg, D. A. Schult, and P. J. Swart, in *Proceedings of the 7th Python in Science Conference*, edited by G. Varoquaux, T. Vaught, and J. Millman (Pasadena, CA USA, 2008) pp. 11–15.

Appendix A: Power-Law Degree Distribution

Our derivation follows that of He et al. [25], which in turn is based on a derivation by Mitzenmacher [31].

At stationarity, the mean degree $\langle d \rangle$ of a node satisfies

$$\langle d \rangle = \frac{m}{n} \langle k \rangle \ , \tag{A1}$$

where $\langle k \rangle$ is the mean edge size (computable from the stationary distribution $p_{K_0,K_1}(k_1, k_2)$), m is the number of edges, and n is the number of nodes. In expectation, there are $a \triangleq a_- + a_+$ new nodes added per new edge, which means that $\frac{m}{n} \to \frac{1}{a}$ as $m, n \to \infty$ in expectation. So, we have

$$\langle d \rangle = \frac{\langle k \rangle}{a} \ . \tag{A2}$$

To proceed, we make a mean-field assumption that the degree d of a node is independent of the ratio $\frac{k_0}{k}$ of label-0 nodes in the edge f sampled to form the new edge e. Define the moments

$$\rho_{ij} = \left\langle \frac{k_i k_j}{k} \right\rangle \ . \tag{A3}$$

We'll first calculate the expected number of nodes of label 0 sampled in the edge-sampling step. Suppose that an edge f is sampled with K_0 nodes of label 0 and K_1 nodes of label 1, and $K_0 + K_1 = K$ total nodes. Conditional on this event, the probability that a node of label 0 is selected as the seed node u is $\frac{K_0}{K}$, and when this occurs, the expected number of nodes of label 0 in e is $1+p_+(K_0-1)$. If, on the other hand, a node of label 1 is selected as the seed node u, which occurs with probability $\frac{K_1}{K}$, then the expected number of nodes of label 0 in e is $p_- K_0$. So, the expected number of nodes of label 0 copied into the new edge e from f is

$$\sigma_0 \triangleq \left\langle \frac{k_0}{k} \left(1 + p_+(k_0 - 1)\right) + \frac{k_1}{k} p_- k_0 \right\rangle \tag{A4}$$

$$= 1 + p_+ \left[\rho_{00} + \rho_{11} - 1\right] + p_- \rho_{01} \ . \tag{A5}$$

Similarly, the expected number of nodes of label 1 copied into the new edge e from f is

$$\sigma_1 \triangleq \left\langle \frac{k_1}{k}\left(1 + p_+(k_1 - 1)\right) + \frac{k_0}{k} p_- k_1 \right\rangle \tag{A6}$$

$$= 1 + p_+\left[\rho_{00} + \rho_{11} - 1\right] + p_- \rho_{01} \; . \tag{A7}$$

We let

$$\sigma \triangleq \sigma_0 + \sigma_1 = 1 + p_+(\rho_{00} + \rho_{11} - 1) + 2p_-\rho_{01} \; . \tag{A8}$$

Importantly, σ depends only on the parameters and the moments of the joint distribution $p_{K_0,K_1}(k_1, k_2)$, which can be computed via linear algebra as described in Section II A 1.

We now invoke the mean-field assumption. Under this assumption, the probability that a single node selected in the sampling step has degree d is $dp_d^{(t)}$, where $p_d^{(t)}$ is the fraction of nodes with degree d at time t, since that node participates in d edges, independently of the ratio $\frac{k_0}{k}$ of label-0 nodes in the edge f sampled to form the new edge e. The expected number of nodes of degree d sampled in the edge-sampling step is then $\sigma \frac{dp_d^{(t)}}{\langle d \rangle}$, and the expected *change* in the count of degree d nodes through the edge-sampling step is

$$\frac{\sigma}{\langle d \rangle}\left((d-1)p_{d-1}^{(t)} - dp_d^{(t)}\right) \tag{A9}$$

Similarly, the expected number of nodes which are sampled in the extant node addition step is $r \triangleq r_+ + r_-$. Since this step does not involve any selection proportional to degree, the expected change in the count of degree d nodes through this step is

$$n^{(t+1)}p_d^{(t+1)} - n^{(t)}p_d^{(t)} = \frac{\sigma}{\langle d \rangle}\left[(d-1)p_{d-1}^{(t)} - dp_d^{(t)}\right] + r(p_{d-1}^{(t)} - p_d^{(t)}) \; . \tag{A10}$$

We know that at stationarity, $n^{(t+1)} - n^{(t)}$ is in expectation equal to $a \triangleq a_+ + a_-$, the expected number of novel nodes, while at stationarity $p_d^{(t+1)} = p_d^{(t)} = p_d$ independent of t. Thus, we obtain

$$p_d a = \frac{\sigma}{\langle d \rangle}\left[(d-1)p_{d-1} - dp_d\right] + r(p_{d-1} - p_d) \; .$$

Rearranging, we obtain

$$\frac{p_d}{p_{d-1}} = \frac{\frac{\sigma}{\langle d \rangle}(d-1) + r}{\frac{\sigma}{\langle d \rangle}d + a + r}$$

$$= 1 - \frac{\frac{\sigma}{\langle d \rangle} + a}{\frac{\sigma}{\langle d \rangle}d + a + r} \; .$$

Allowing d to become large (which describes the tail of the degree distribution), we have

$$\frac{p_d}{p_{d-1}} \approx 1 - \frac{1}{d}\frac{\frac{\sigma}{\langle d \rangle} + a}{\frac{\sigma}{\langle d \rangle}}$$

$$= 1 - \frac{1}{d}\left(1 + \frac{a}{\sigma / \langle d \rangle}\right)$$

$$= 1 - \frac{1}{d}\left(1 + \frac{\langle k \rangle}{\sigma}\right) \; .$$

a relation which corresponds to a power law with exponent $\zeta = 1 + \frac{\langle k \rangle}{\sigma}$. Explicitly calculating this exponent requires the moments $\langle k \rangle$, ρ_{00}, ρ_{11}, and ρ_{01}, which can be computed from the stationary joint distribution $p_{K_0,K_1}(k_1, k_2)$.

Appendix B: Sufficient Statistics for Stochastic Expectation-Maximization

We describe the function σ from Section III A which computes the conditional sufficient statistics for the parameters $\boldsymbol{\theta} = (p_+, p_-, r_+, r_-, a_+, a_-)$ given an observed edge e, a candidate parent edge f, a candidate seed node u, and the node labels $\mathbf{z}$. Let $z = z_u$ be the label of the seed node u. Let e_z be the set of nodes in e with label z, and let f_z be the set of nodes in f with label z. Then, the sufficient statistics for the parameters $\boldsymbol{\theta}$ are given by

$$\sigma(e, f, u, \mathbf{z}) = \begin{pmatrix} |e_z \cap f_z| \\ |f_z| \\ |e_{\bar{z}} \cap f_{\bar{z}}| \\ |f_{\bar{z}}| \\ \left|(e_z \setminus f_z) \cap \mathcal{V}^{(t)}\right| \\ \left|(e_{\bar{z}} \setminus f_{\bar{z}}) \cap \mathcal{V}^{(t)}\right| \\ \left|e_z \setminus \mathcal{V}^{(t)}\right| \\ \left|e_{\bar{z}} \setminus \mathcal{V}^{(t)}\right| \end{pmatrix}^\top \; .$$

As descrbed in Section III A, the vector $\mathbf{s}$ of expected sufficient statistics is computed as a weighted average of $\sigma(e, f, u, \mathbf{z})$ over the belief distribution $p(f, u | e, \hat{\boldsymbol{\theta}}^{(\ell)})$ over the latent variables f and u. Then, the function g which computes the maximum likelihood estimates of the parameters $\boldsymbol{\theta}$ from $\mathbf{s}$ is given by

$$g(\mathbf{s}) = \left(\frac{s_1 - 1}{s_2 - 1}, \frac{s_3}{s_4}, s_5, s_6, s_7, s_8\right) \; .$$

PC: Violet, I added the -1 here to what you had previously written but please check.

Appendix C: Approximation Argument for Simulated Anealing

This section provides further detail on the approximation to the likelihood described in Section III B and used in our simulated annealing implementation. The likelihood of an edge e being generated in timestep t can be written

$$\mathbb{P}(e|\mathbf{z};\boldsymbol{\theta}) = \sum_{\substack{f\prec e\\ u\in f}} \mathbb{P}(e|f,u,\mathbf{z};\boldsymbol{\theta}) \tag{C1}$$

$$= \frac{1}{m_t}\sum_{f\prec e}\frac{1}{|f|}\sum_{u\in f\cap e}\mathbb{P}(e|f,u,\mathbf{z};\boldsymbol{\theta})\ , \tag{C2}$$

where m_t is the number of edges at time t. It's helpful to reindex this sum by the size of the intersection $|f\cap e|$ between f and e.

$$\mathbb{P}(e|\mathbf{z};\boldsymbol{\theta}) = \sum_{h=0}^{|e|}\sum_{\substack{f\prec e\\ |f\cap e|=k}}\frac{1}{m_t}\sum_{u\in f\cap e}\frac{1}{|f|}\mathbb{P}(e|f,u,\mathbf{z};\boldsymbol{\theta})\ . \tag{C3}$$

We'll now argue that, when n is large, the terms in this sum corresponding to the largest intersections (for which $h = \text{argmax}_{f\prec e}|f\cap e|$) will dominate the likelihood, allowing us to neglect edges f whose intersection with e is smaller.

Fix edge e and candidate parent edge f. We subscript e_z, f_z, and $\mathcal{V}_z$ to denote the subsets of e, f, and $\mathcal{V}$ with label z. Let $h_z = |(e_z\setminus f_z)\cap\mathcal{V}_z|$ be the number of extant nodes of label z in e. Then, the extant node addition process appears in $\mathbb{P}(e|f,u,\mathbf{z};\boldsymbol{\theta})$ as a factor

$$E(h_0,h_1) = \frac{\text{Poisson}(h_{z_u}|r_+)\times\text{Poisson}(h_{\bar{z}_u}|r_-)}{\binom{|\mathcal{V}_0^t|-|f_0|}{h_0}\binom{|\mathcal{V}_1^t|-|f_1|}{h_1}}\ , \tag{C4}$$

where the numerator describes the likelihood of adding fixed numbers of extant nodes of each label and the denominator describes the likelihood of selecting the *specific set* of extant nodes from the set of all candidates. Under the assumption that $n\gg|f|,|e|$, we can estimate

$$\binom{|\mathcal{V}_z^t|-|f_z|}{h_z} \approx \frac{(|\mathcal{V}_z^t|)^{h_z}}{h_z!}\approx\frac{(c_z n)^{h_z}}{h_z!}\ , \tag{C5}$$

where we have defined $c_z = \frac{|\mathcal{V}_z^t|}{n}$ as the fraction of nodes with label z. Then, absorbing terms that do not depend on n into a constant term $C(h_0,h_1)$, we have

$$E(h_0,h_1) = C(h_0,h_1)n^{-h_0-h_1}+O(n^{-h_0-h_1-1}) \tag{C6}$$

$$= C(h_0,h_1)n^{-h}+O(n^{-h-1})\ , \tag{C7}$$

where $h = h_0+h_1 = |(e\setminus f)\cap\mathcal{V}|$ is the total number of extant nodes added to e and

$$C(h_0,h_1) = \frac{\text{Poisson}(h_{z_u}|r_+)\times\text{Poisson}(h_{\bar{z}_u}|r_-)}{\frac{(c_0)^{h_0}}{h_0!}\frac{(c_1)^{h_1}}{h_1!}}\ . \tag{C8}$$

It follows that $\mathbb{P}(e|f,u,\mathbf{z};\boldsymbol{\theta})$ is of order $O(n^{-h})$ as n grows large, where h is the number of extant nodes added to e. Terms corresponding to choices of f for which h is larger (of which there may be up to approximately m) decay quickly for large n. We therefore have

$$\mathbb{P}(e|\mathbf{z};\boldsymbol{\theta}) = \sum_{\substack{f\prec e\\ |f\cap e|=h'}}\frac{1}{m_t}\sum_{u\in f\cap e}\frac{1}{|f|}\mathbb{P}(e|f,u,\mathbf{z};\boldsymbol{\theta}) + mO(n^{-h'-1}) \tag{C9}$$

where $h' = \min_{f\prec e}|(e\setminus f)\cap\mathcal{V}|$ is the minimum number of extant nodes added to e across all choices of f.

This argument suggests that, when n is relatively large in relation to m, the likelihood of e is dominated by terms corresponding to choices of f for which the number of extant nodes added to e is small. There are two limitations to this argument in the context of empirical data sets. First, it may be the case that only a small number of candidate parent edges f have small h, which may lead to a poor approximation of the likelihood when we evaluate only a small number of corresponding terms. Second, several of our data sets have m large in relation to n, making it dubious to neglect the error term. We therefore introduce two variations to our approximation heuristic:

- **Top j heuristic**: Rather than evaluate only terms for f for which h is minimal, we evaluate terms for the top j choices of f with the smallest h.
- **Batch normalization**: Rather than normalizing by $\frac{1}{m_t}$ in eq. (C9), we normalize by the number of f choices we evaluate for a specific e. This may be j under the top j heuristic, or it may be the number of f choices with h equal to the minimum h' if we do not use the top j heuristic. Letting $\mathcal{F}_e$ be the set of f choices we evaluate for a specific e, our approximation is

$$\tilde{\mathbb{P}}(e|\mathbf{z};\boldsymbol{\theta}) = \frac{1}{|\mathcal{F}_e|}\sum_{f\in\mathcal{F}_e}\sum_{u\in f\cap e}\frac{1}{|f|}\mathbb{P}(e|f,u,\mathbf{z};\boldsymbol{\theta})\ . \tag{C10}$$

Meaning the approximate likelihood of the entire $\mathcal{H}$ is

$$\tilde{\mathbb{P}}(\mathcal{H},\mathbf{z};\boldsymbol{\theta}) = \prod_{e\in\mathcal{E}}\tilde{\mathbb{P}}(e|\mathbf{z};\boldsymbol{\theta}) \tag{C11}$$

It is important to note that the batch normalization eq. (C11) tends to overestimate the likelihood, since we have restricted ourselves to an average of high-probability seed-edges. For the purposes of simulated annealing, this is not a major problem since we are only interested in the relative likelihood of different labelings $\mathbf{z}$ and not the absolute likelihood.

Appendix D: Simulated Annealing for Community Detection

We describe the simulated annealing algorithm implementation in detail. Define the number of epochs for the simulated annealing algorithm $A = 20$. Let B be the probability that the simulated annealing algorithm accepts a change on labels that lowers the approximated likelihood

Algorithm 2 `SimulatedAnnealing`$(\mathcal{H}, \theta)$

Require: Hypergraph $\mathcal{H}$, parameters $\boldsymbol{\theta}$, number of epochs A
Ensure: Label set $\mathbf{z}$, approximation of the true labels $\mathbf{z}^*$

```
Sample z ~ Uniform({0,1})^n
ΔL ← ∅
σ_H ← 0
for t = 0 to A do
    for τ = 0 to n do
        Sample j ~ Uniform({0, ..., n−1})
        z' ← z
        z'_j ← 1 − z'_j
        ΔE ← log P̃(H, z'|θ) − log P̃(H, z|θ)
        if t = 0 then
            ΔL ← ΔL ∪ {ΔE}
        if σ_H = 0 & t > 0 then
            σ_H ← σ(ΔL)
        if ΔE > 0 then
            z ← z'
        else
            B ← 1
            ζ_a ← ΔE/σ_H
            if 1 ≤ t ≤ 4 then
                Sample B ~ Normal(ζ_a|μ = 0, σ = 2)
            else if 5 ≤ t then
                Sample B ~ Normal(ζ_a|μ = 0, σ = 2(1 − t/A))
            Sample x ~ Uniform(0, 1)
            if B > x then
                z ← z'
return z
```

Here,

$$\sigma(\Delta\mathcal{L}) = \sqrt{\frac{\sum_{\delta\in\Delta\mathcal{L}} \delta^2 - (\frac{1}{|\Delta\mathcal{L}|}\sum_{\delta\in\Delta\mathcal{L}} \delta)^2}{|\Delta\mathcal{L}| - 1}} \tag{D1}$$

Appendix E: Simulated Annealing Illustration

In this simulation, we generated a simple hypergraph according to our model with 65 nodes and 200 edges. Running simulated annealing for 1300 steps, we obtained a maximum log-likelihood of -9.70 nats per edge with an Adjusted Rand Index (ARI) against ground-truth of 0.47. Greedy modularity maximization as implemented in the Python package `networkx` [41] obtained an ARI of -0.00. The parameters for the model used to generate the hypergraph are

$$\begin{aligned} p_+ &= 0.90 \\ p_- &= 0.10 \\ r_+ &= 1.00 \\ r_- &= 0.25 \\ a_+ &= 0.20 \\ a_- &= 0.10\ . \end{aligned}$$

The guessed parameters used to compute the likelihood during simulated annealing are

$$\begin{aligned} \hat{p}_+ &= 0.90 \\ \hat{p}_- &= 0.10 \\ \hat{r}_+ &= 1.00 \\ \hat{r}_- &= 0.25 \\ \hat{a}_+ &= 0.20 \\ \hat{a}_- &= 0.10\ . \end{aligned}$$

Appendix F: Gradient Descent Illustration

TODO

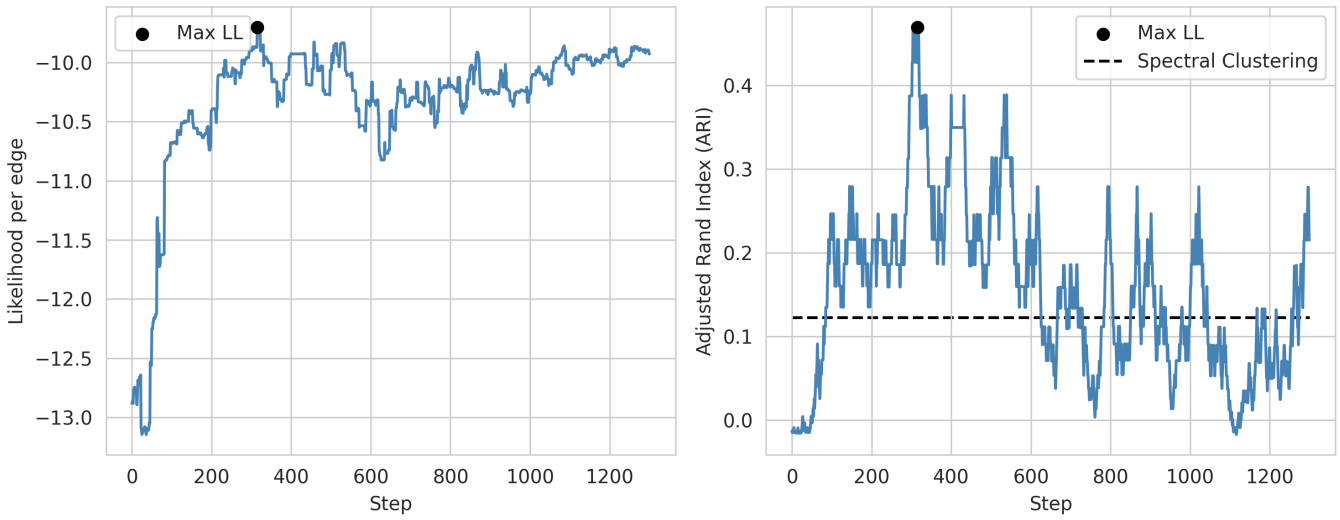

FIG. 5. Illustrative run of simulated annealing for community detection in a hypergraph generated from our model.